

Rover Ready: A WebXR Rover Repair Bay

Chirag Baviskar Dhrumil Patel
Rutgers University
Virtual Reality & Lab, Project 2

Abstract—Rover Ready is a WebXR repair-bay demo built around one small job: get a rover ready to leave the garage. The player calls the rover in, installs its power cell, repairs marked components with a handheld tool, replaces missing parts, runs a check, and sends the rover back out through the bay door. We kept the scene to a single garage so most of the work could go into the interaction loop instead of empty extra space. The final build uses Three.js, WebXR, GLTF-loaded Blender models, and Cannon-ES physics for the objects the player can grab, move, and snap into place.

Index Terms—WebXR, Three.js, Blender, Cannon-ES, GLTFLoader, virtual reality

I. PROJECT OVERVIEW

Rover Ready is our final project for Rutgers Virtual Reality & Lab Project 2. The demo puts the player in a low-poly rover garage on a Mars service site. The room is small by design. There is a rover lane, a tool table, a power-cell rack, a diagnostic display, and a bay door. Everything in the scene points back to the same task: service the rover and clear it for launch.

The project is not a walkthrough scene. The rover enters the bay, parks in front of the player, and waits for a checklist of repairs. The player can grab loose objects, snap parts onto the rover, aim a repair tool at damaged components, and use the in-scene buttons to call, check, and send rovers. The later levels reuse that same layout but ask for more work, including a larger rover with two power slots.

The demo uses Three.js and WebXR for the browser runtime. GLTFLoader loads the garage bay, regular rover, complex rover, power cell, and repair tool as separate GLB files. Those visible models were made for this project in Blender. Cannon-ES is used for the objects that need to behave like loose physical items, mainly the power cells, the handheld tool, and removable rover parts.

The two advanced features we count are physics and multi-object modes. Physics makes the repair objects feel separate from the static room instead of being simple click targets. Multiple object modes show up in the way the demo treats buttons, power cells, repair-tool aiming, loose parts, regular rovers, complex rovers, and the level sequence differently.

II. SCENE AND INTERACTION DESCRIPTION

The player starts inside the garage facing the rover lane. The left side of the bay holds the tool table and loose parts. The control panel has three important buttons: call, check, and send. A Mars exterior is visible past the door, which helps the bay feel like part of a larger site without making the user travel through a second area.

The first action is to press the call button. The door opens, the rover drives in from outside, and it turns into its parked orientation. For a power task, the player grabs a battery from the rack and moves it toward the rover’s receiver. It remains a physics object while it is being carried. When its insert point is close enough to the rover’s snap target, the module locks into the dock and the checklist updates.

The repair-tool task works differently. The player picks up the tool and points its tip at a damaged component. The target highlights while the tool is aligned, then completes after being held there long enough. The missing-part tasks add a third behavior. A part such as a side panel, wheel, headlamp, bumper, antenna, or science pod appears on the table, and the player has to place it near the matching snap point on the rover.

The check button is intentionally part of the loop. It prevents the player from sending a rover that is not finished and gives a clear next objective when something is still missing. When the checklist passes, the send button is enabled, the door opens, and the rover exits the bay. Level one focuses on the power cell, level two introduces tool repair, levels three through five combine regular-rover tasks, and level six switches to the complex rover with two power slots and a larger set of possible part repairs.

Demo video link: [Rover Ready demo video](#).

III. FEATURES TABLE

Table I maps the assignment requirements to parts of the final demo. The base requirements are listed separately from the advanced features.

IV. CHALLENGES AND SOLUTIONS

The hardest part was keeping Blender and the browser code in agreement. It was not enough for a model to look correct in Blender. The runtime also needed to know where the power cell should snap, where the repair tool should aim, which mesh belonged to a removable part, and how large the physics body should be. We handled this by giving gameplay nodes stable names in the Blender exports, such as `RR_PowerCell_InsertPoint` and component `_SnapTarget` nodes, then reading those names from JavaScript.

Physics needed a similar compromise. The visible rover and bay meshes are too detailed to use directly as collision geometry. When we tried to treat visual shape and gameplay shape as the same thing, small objects became harder to grab and place. The final version uses simpler Cannon-ES boxes

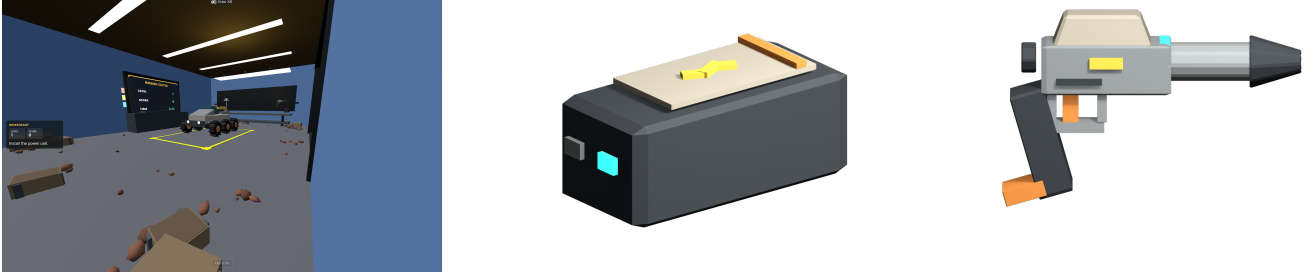


Fig. 1. Final demo visuals. Left: the rover parked inside the repair bay. Center: the custom power-cell module. Right: the custom handheld repair tool.

TABLE I
REQUIREMENT COVERAGE

Req.	Type	Coverage	Section
Original theme	Base	Rover service bay	Overview
Custom 3D assets	Base	Bay, rovers, cell, tool	Assets
Core interactions	Base	Power, repair, parts, buttons	Scene
Physics	Category A	Grabbable cells, tool, parts	Challenges
Object types/modes	Category A	Buttons, tools, parts, rovers	Scene
Blender generation	Supporting	Python-built project assets	Execution

for the table, power-cell body, repair tool, and loose parts. The GLB meshes still provide the visual detail, while the physics shapes keep the interaction predictable.

The rover flow also needed more state control than expected. Calling, parking, checking, opening the door, and sending the rover all touch the same model and the same collision bodies. The final code separates arrival, turning, repair, departure turning, and departure movement so the rover does not fight its static bodies or leave before the door sequence is ready.

Scope was the last major issue. A larger Mars base would have made the project look bigger but would not have improved the required interactions. We kept the player in one bay and spent the extra time on snap targets, repair feedback, button states, and the later complex-rover levels.

V. DISCUSSION AND FUTURE WORK

The best part of the final demo is that the loop is easy to understand without needing a separate tutorial. The rover stops in a consistent place, the loose parts spawn on the table, and the buttons control obvious bay actions. The snap system also helps with usability. The player still has to move the object to the right area, but the final alignment does not require unrealistic hand precision.

The roughest part is still feedback polish. The HUD, glowing targets, snapped objects, and button lights communicate the state of the rover, but the repair tool would feel better with sound, haptics, and a more active animation. The diagnostic screen could also do more than show status text. It could highlight a diagram of the rover and point to the next damaged part.

If we continued the project, we would add those feedback pieces before adding a new room. More repair job types would

also fit, but they should stay inside the same bay structure. The project works best when the player is focused on the rover instead of searching through a larger environment.

VI. INDIVIDUAL CONTRIBUTIONS

Table II lists the main ownership split. The percentages are approximate and describe who led each area rather than exact time spent.

VII. ASSET CREDITS

The garage bay, regular rover, complex rover, power-cell module, repair tool, and Mars exterior were made for this project in Blender. The source submission includes the original .blend files, exported .glb runtime files, and the Blender Python scripts used to generate the assets.

The runtime depends on Three.js for rendering and WebXR support, GLTFLoader for model loading, and Cannon-ES for physics. These are software libraries, not third-party scene assets. The final visible scene does not use outside 3D models, outside texture packs, 360-degree media, or Gaussian splats.

TABLE II
INDIVIDUAL CONTRIBUTION SUMMARY

Component	Lead	Approx. Share
Asset visual direction	Chirag Baviskar	60% / 40%
Asset export/package	Chirag Baviskar	70% / 30%
Three.js/WebXR runtime	Dhruvil Patel	30% / 70%
Input handling	Dhruvil Patel	30% / 70%
Physics and collisions	Dhruvil Patel	30% / 70%
Game flow and HUD	Dhruvil Patel	35% / 65%
Final docs/presentation	Chirag Baviskar	60% / 40%

VIII. CODE AND EXECUTION INSTRUCTIONS

After extracting the source zip, use the extracted folder as the project root. All paths in these instructions are relative to that folder. The runnable entrypoint is `./index.html`. The demo needs a local web server because it uses JavaScript modules, CDN imports, and GLB asset loading. Opening `./index.html` directly from the file system will not load the demo correctly.

From the extracted source folder, run one of the included scripts:

```
.\run-demo.ps1
run-demo.bat
./run-demo.sh
```

Each script serves the project at:

```
http://localhost:5173/
```

The same server can be started manually from that folder:

```
python -m http.server 5173
```

Then open `http://localhost:5173/` in a Chromium-based browser. Desktop controls are included for testing: W, A, S, and D move the player, the arrow keys turn, and mouse dragging grabs objects. In VR, the controller thumbsticks handle movement and trigger/select grabs objects. The WebXR Emulator browser extension can be used when a headset is not available.

The source materials include `./index.html`, the JavaScript files under `./src`, runtime GLB files under `./src/assets/models`, Blender source files under `./src/assets/blender`, and Blender generation scripts under `./scripts/blender`.

REFERENCES

- [1] Three.js, “Three.js Documentation,” <https://threejs.org/docs/>, accessed: 2026-05-04.
- [2] MDN Web Docs, “WebXR Device API,” https://developer.mozilla.org/en-US/docs/Web/API/WebXR_Device_API, accessed: 2026-05-04.
- [3] pmndrs, “Cannon-ES Documentation,” <https://pmndrs.github.io/cannon-es/docs/>, accessed: 2026-05-04.
- [4] Blender Foundation, “Blender Python API Documentation,” <https://docs.blender.org/api/current/>, accessed: 2026-05-04.